UNIT-I

The Reinforcement Learning Problem: Reinforcement Learning, Examples, Elements of Reinforcement Learning, Limitations and Scope, An Extended Example: Tic-Tac-Toe, History of Reinforcement Learning.

C. JYOTHSNA, Assistant Professor, Dept of CSE, NBKRIST

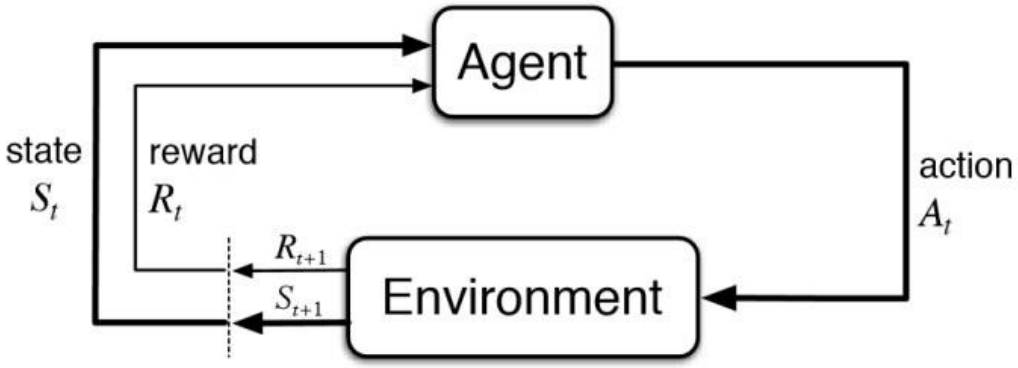
1. Reinforcement Learning

- Reinforcement learning is learning what to do—how to map situations to actions—so as to maximize a numerical reward signal.
- The learner is not told which actions to take, but instead must discover which actions yield the most reward by trying them.
- In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situation and, through that, all subsequent rewards.
- These two characteristics—trial-and-error search and delayed reward—are the two most important distinguishing features of reinforcement learning.
- We formalize the problem of reinforcement learning using ideas from dynamical systems theory, specifically, as the optimal control of incompletely-known Markov decision processes.
- A learning agent must be able to sense the state of its environment to some extent and must be able to take actions that affect the state.
- The agent also must have a goal or goals relating to the state of the environment.

- Markov decision processes are intended to include just these three aspects—sensation, action, and goal—in their simplest possible forms without trivializing any of them. Any method that is well suited to solving such problems we consider to be a reinforcement learning method.
- Reinforcement learning is different from supervised learning, the kind of learning studied in most current research in the field of machine learning. Supervised learning is learning from a training set of labelled examples provided by a knowledgeable external supervisor.
- The object of this kind of learning is for the system to extrapolate, or generalize, its responses so that it acts correctly in situations not present in the training set.
- Reinforcement learning is also different from what machine learning researchers call unsupervised learning, which is typically about finding structure hidden in collections of unlabelled data.
- The agent has to exploit what it has already experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.
- The dilemma is that neither exploration nor exploitation can be pursued exclusively without failing at the task. The agent must try a variety of actions and progressively favour those that appear to be best.
- On a stochastic task, each action must be tried many times to gain a reliable estimate of its expected reward.

C. JYOTHSNA, Assistant Professor, Dept of CSE, NBKRIST

- The exploration–exploitation dilemma has been intensively studied by mathematicians for many decades, yet remains unresolved. For now, we simply note that the entire issue of balancing exploration and exploitation does not even arise in supervised and unsupervised learning, at least in the purest forms of these paradigms.
- Another key feature of reinforcement learning is that it explicitly considers the whole problem of a goal-directed agent interacting with an uncertain environment.
- Reinforcement learning takes the opposite tack, starting with a complete, interactive, goal-seeking agent. All reinforcement learning agents have explicit goals, can sense aspects of their environments, and can choose actions to influence their environments.



Basic Diagram of Reinforcement Learning

Reinforcement Learning is a part of machine learning. Here, agents are self-trained on reward and punishment mechanisms. It's about taking the best possible action or path to gain maximum rewards and minimum punishment through observations in a specific situation. It acts as a signal to positive and negative behaviours. Essentially an agent (or several) is built that can perceive and interpret the environment in which is placed, furthermore, it can take actions and interact with it.

C. JYOTHSNA Assistant Professor, NBKRIST

2. Examples

A good way to understand reinforcement learning is to consider some of the examples and possible applications that have guided its development.

- A master chess player makes a move. The choice is informed both by planning— anticipating possible replies and counter replies—and by immediate, intuitive judgments of the desirability of particular positions and moves.
- An adaptive controller adjusts parameters of a petroleum refinery's operation in real time. The controller optimizes the yield/cost/quality trade-of on the basis of specified marginal costs without sticking strictly to the set points originally suggested by engineers.
- A gazelle calf struggles to its feet minutes after being born. Half an hour later it is running at 20 miles per hour.
- A mobile robot decides whether it should enter a new room in search of more trash to collect or start trying to find its way back to its battery recharging station. It makes its decision based on the current charge level of its battery and how quickly and easily it has been able to find the recharger in the past.
- Phil prepares his breakfast. Closely examined, even this apparently mundane activity reveals a complex web of conditional behaviour and interlocking goal–sub goal relationships: walking to the cupboard, opening it, selecting a cereal box, then reaching for, grasping, and retrieving the box. Other complex, tuned, interactive sequences of behaviour are required to obtain a bowl, spoon, and milk carton. Each step involves a series of eye movements to obtain information and to guide reaching and locomotion. Rapid judgments are continually made about how to carry the objects or whether it is better to ferry some of them to the dining table before obtaining others. Each step is guided by goals, such as grasping a spoon or getting to the refrigerator, and is in service of other goals, such as having the spoon to eat with once the cereal is prepared and ultimately obtaining nourishment. Whether he is aware of it or not, Phil is accessing information about the state of his body that determines his nutritional needs, level of hunger and food preferences.

C. JYOTHSNA, Assistant Professor, Dept of CSE, NBKRIST

3. Elements of Reinforcement Learning

Beyond the agent and the environment, one can identify four main sub elements of a reinforcement learning system:

- a policy,
- a reward signal,
- a value function, and, optionally,
- a model of the environment.

POLICY:

- A policy defines the learning agent's way of behaving at a given time.
- Roughly speaking, a policy is a mapping from perceived states of the environment to actions to be taken when in those states.
- It corresponds to what in psychology would be called a set of stimulus–response rules or associations.
- In some cases the policy may be a simple function or lookup table, whereas in others it may involve extensive computation such as a search process.
- The policy is the core of a reinforcement learning agent in the sense that it alone is sufficient to determine behaviour.
- In general, policies may be stochastic, specifying probabilities for each action.

REWARD SIGNAL:

- A reward signal defines the goal of a reinforcement learning problem.
- On each time step, the environment sends to the reinforcement learning agent a single number called the reward.
- The agent's sole objective is to maximize the total reward it receives over the long run.
- The reward signal thus defines what the good and bad events are for the agent.
- In a biological system, we might think of rewards as analogous to the experiences of pleasure or pain.
- They are the immediate and defining features of the problem faced by the agent.
- The reward signal is the primary basis for altering the policy; if an action selected by the policy is followed by low reward, then the policy may be changed to select some other action in that situation in the future.
- In general, reward signals may be stochastic functions of the state of the environment and the actions taken.

VALUE FUNCTION:

- A value function specifies what is good in the long run.
- Roughly speaking, the value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state.

C. JYOTHSNA, Assistant Professor, Dept of CSE, NBKRIST

- Whereas rewards determine the immediate, intrinsic desirability of environmental states, values indicate the long-term desirability of states after taking into account the states that are likely to follow and the rewards available in those states.
- For example, a state might always yield a low immediate reward but still have a high value because it is regularly followed by other states that yield high rewards. Or the reverse could be true.
- To make a human analogy, rewards are somewhat like pleasure (if high) and pain (if low), whereas values correspond to a more refined and farsighted judgment of how pleased or displeased we are that our environment is in a particular state.

MODEL OF THE ENVIRONMENT:

- The fourth and final element of some reinforcement learning systems is a model of the environment.
- This is something that mimics the behaviour of the environment, or more generally, that allows inferences to be made about how the environment will behave.
- For example, given a state and action, the model might predict the resultant next state and next reward.
- Models are used for planning, by which we mean any way of deciding on a course of action by considering possible future situations before they are actually experienced.
- Methods for solving reinforcement learning problems that use models and planning are called model-based methods, as opposed to simpler model-free methods that are explicitly trial-and-error learners—viewed as almost the opposite of planning.

C. JYOTHSNA, Assistant Professor, Dept of CSE, NBKRIST

4. Limitations and Scope

- Reinforcement learning relies heavily on the concept of state—as input to the policy and value function, and as both input to and output from the model.
- Informally, we can think of the state as a signal conveying to the agent some sense of "how the environment is" at a particular time.
- The formal definition of state as we use it here is given by the framework of Markov decision processes.
- More generally, however, we encourage the reader to follow the informal meaning and think of the state as whatever information is available to the agent about its environment.
- In effect, we assume that the state signal is produced by some pre-processing system that is nominally part of the agent's environment.
- We take this approach not because we consider state representation to be unimportant, but in order to focus fully on the decision-making issues.

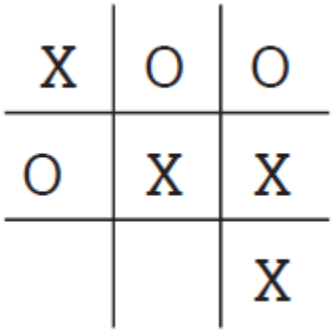
- Most of the reinforcement learning methods are structured around estimating value functions, but it is not strictly necessary to do this to solve reinforcement learning problems.
- For example, solution methods such as genetic algorithms, genetic programming, simulated annealing, and other optimization methods never estimate value functions. These methods apply multiple static policies each interacting over an extended period of time with a separate instance of the environment.
- The policies that obtain the most reward, and random variations of them, are carried over to the next generation of policies, and the process repeats.
- We call these evolutionary methods because their operation is analogous to the way biological evolution produces organisms with skilled behaviour even if they do not learn during their individual lifetimes. If the space of policies is sufficiently small.
- In addition, evolutionary methods have advantages on problems in which the learning agent cannot sense the complete state of its environment.
- Our focus is on reinforcement learning methods that learn while interacting with the environment, which evolutionary methods do not do.

C. JYOTHSNA, Assistant Professor, Dept of CSE, NBKRIST

5. An Extended Example: Tic-Tac-Toe

To illustrate the general idea of reinforcement learning and contrast it with other approaches, we next consider a single example in more detail.

- Consider the familiar child's game of tic-tac-toe.
- Two players take turns playing on a three-by-three board.
- One player plays Xs and the other Os until one player wins by placing three marks in a row, horizontally, vertically, or diagonally, as the X player has in the game shown to the right.
- If the board fills up with neither player getting three in a row, then the game is a draw. Because a skilled player can play so as never to lose, let us assume that we are playing against an imperfect player, one whose play is sometimes incorrect and allows us to win.
- For the moment, in fact, let us consider draws and losses to be equally bad for us.
- How might we construct a player that will find the imperfections in its opponent's play and learn to maximize its chances of winning?



- Although this is a simple problem, it cannot readily be solved in a satisfactory way through classical techniques.
- For example, the classical "minimax" solution from game theory is not correct here because it assumes a particular way of playing by the opponent.
- For example, a minimax player would never reach a game state from which it could lose, even if in fact it always won from that state because of incorrect play by the opponent.
- Classical optimization methods for sequential decision problems, such as dynamic programming, can compute an optimal solution for any opponent, but require as input a complete specification of that opponent, including the probabilities with which the opponent makes each move in each board state.
- Let us assume that this information is not available a priori for this problem, as it is not for the vast majority of problems of practical interest.

C. JYOTHSNA, Ass
Dept of CSE, NBKRIST

- On the other hand, such information can be estimated from experience, in this case by playing many games against the opponent.
- About the best one can do on this problem is first to learn a model of the opponent's behaviour, up to some level of confidence, and then apply dynamic programming to compute an optimal solution given the approximate opponent model.
- An evolutionary method applied to this problem would directly search the space of possible policies for one with a high probability of winning against the opponent.
- Here, a policy is a rule that tells the player what move to make for every state of the game—every possible configuration of Xs and Os on the three-by-three board.
- For each policy considered, an estimate of its winning probability would be obtained by playing some number of games against the opponent.
- This evaluation would then direct which policy or policies were considered next.
- A typical evolutionary method would hill-climb in policy space, successively generating and evaluating policies in an attempt to obtain incremental improvements. Or, perhaps, a genetic-style algorithm could be used that would maintain and evaluate a population of policies.
- Literally hundreds of different optimization methods could be applied.
- Here is how the tic-tac-toe problem would be approached with a method making use of a value function.
- First we would set up a table of numbers, one for each possible state of the game. Each number will be the latest estimate of the probability of our winning from that state.
- We treat this estimate as the state's value, and the whole table is the learned value function.
- State A has higher value than state B, or is considered "better" than state B, if the current estimate of the probability of our winning from A is higher than it is from B. Assuming we always play Xs, then for all states with three Xs in a row the probability of winning is 1, because we have already won.
- Similarly, for all states with three Os in a row, or that are filled up, the correct probability is 0, as we cannot win from them. We set the initial values of all the other states to 0.5, representing a guess that we have a 50% chance of winning.
- We then play many games against the opponent. To select our moves we examine the states that would result from each of our possible moves (one for each blank space on the board) and look up their current values in the table. Most of the time we move greedily, selecting the move that leads to the state with greatest value, that is, with the highest estimated probability of winning. Occasionally, however, we select randomly from among the other moves instead. These are called exploratory moves because they cause us to experience states that we might otherwise never see. A sequence of moves made and considered during a game can be diagrammed as in Figure 1.1.

C. JYOTHSNA, Assistant Professor, Dept of CSE, NBKRIST

While we are playing, we change the values of the states in which we find ourselves during the game. We attempt to make them more accurate estimates of the probabilities of winning. To do this, we "back up" the value of the state after each greedy move to the state before the move, as suggested by the arrows in Figure 1.1. More precisely, the current value of the earlier state is updated to be closer to the value of the later state. This can be done by moving the earlier state's value a fraction of the way toward the value of the later state. If we let St denote the state before the greedy move, and St+1 the state after the move, then the update to the estimated value of St, denoted V (St), can be written as

$$V(S_t) \leftarrow V(S_t) + \alpha\Big[V(S_{t+1}) - V(S_t)\Big],$$

NBKRIST

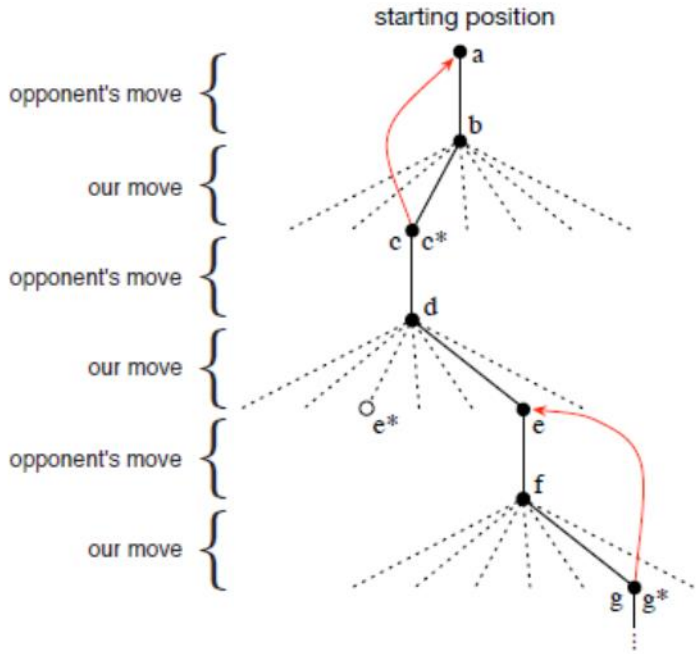


Figure 1.1: A sequence of tic-tac-toe moves. The solid black lines represent the moves taken during a game; the dashed lines represent moves that we (our reinforcement learning player) considered but did not make. Our second move was an exploratory move, meaning that it was taken even though another sibling move, the one leading to e*, was ranked higher. Exploratory moves do not result in any learning, but each of our other moves does, causing updates as suggested by the red arrows in which estimated values are moved up the tree from later nodes to earlier nodes as detailed in the text.

where α is a small positive fraction called the step-size parameter, which influences the rate of learning. This update rule is an example of a temporal-difference learning method, so called because its changes are based on a difference, V (St+1).

6. Early History of Reinforcement Learning

The early history of reinforcement learning has two main threads, both long and rich, that were pursued independently before intertwining in modern reinforcement learning.

One thread concerns learning by trial and error, and originated in the psychology of animal learning.

This thread runs through some of the earliest work in artificial intelligence and led to the revival of reinforcement learning in the early 1980s.

The second thread concerns the problem of optimal control and its solution using value functions and dynamic programming. For the most part, this thread did not involve learning. The two threads were mostly independent, but became interrelated to some extent around a third, less distinct thread concerning temporal-difference methods such as that used in the tic-tac-toe example in this chapter.

All three threads came together in the late 1980s to produce the modern field of reinforcement learning.

The term “optimal control” came into use in the late 1950s to describe the problem of designing a controller to minimize or maximize a measure of a dynamical system’s behaviour over time.

One of the approaches to this problem was developed in the mid-1950s by Richard Bellman and others through extending a nineteenth century theory of Hamilton and Jacobi.

This approach uses the concepts of a dynamical system’s state and of a value function, or “optimal return function,” to define a functional equation, now often called the Bellman equation.

The class of methods for solving optimal control problems by solving this equation came to be known as dynamic programming (Bellman, 1957a). Bellman (1957b) also introduced the discrete stochastic version of the optimal control problem known as Markov decision processes (MDPs).

Ronald Howard (1960) devised the policy iteration method for MDPs. All of these are essential elements underlying the theory and algorithms of modern reinforcement learning.

Dynamic programming is widely considered the only feasible way of solving general stochastic optimal control problems. It suffers from what Bellman called “the curse of dimensionality,” meaning that its computational requirements grow exponentially with the number of state variables, but it is still far more efficient.

C. JYOTHSNA, Assistant Professor, Dept of CSE, NBKRIST

Dynamic programming has been extensively developed since the late 1950s, including extensions to partially observable MDPs (surveyed by Lovejoy, 1991), many applications (surveyed by White, 1985, 1988, 1993), approximation methods (surveyed by Rust, 1996), and asynchronous methods (Bertsekas, 1982, 1983). Many excellent modern treatments of dynamic programming are available (e.g., Bertsekas, 2005, 2012; Puterman, 1994; Ross, 1983; and Whittle, 1982, 1983). Bryson (1996) provides an authoritative history of optimal control.

Analysis:

RL has its origins in animal behaviourism and the study of positive reinforcement by behavioural psychologist B. F. Skinner in the 1930s.

Skinner demonstrated that animals could be trained to perform complex tasks through simple reinforcement mechanisms, such as receiving a food reward for performing a desired control. Skinner conducted several experiments on pigeons and rats in order to study reinforcement. He found that reinforcement could be used to shape animal behaviour, and he developed the concept of positive reinforcement.

The basic idea of positive reinforcement is that an animal or any agent can learn to optimize its behaviour by learning from past experience.

A detailed account of the history of RL is found in Sutton and Barto (1998). Here, we will only summarize the most important contributions.

Most notably there are two:

1. Optimal control
2. Animal learning by trial and error

C. JYOTHSNA, Assistant Professor, Dept of CSE, NBKRIST